当说服式攻击遇上执行层隔离

——Agent 安全的语义完备性困境与物理边界

摘要

大语言模型智能体（LLM Agent）在执行任务时，需要读取外部内容——网页、文档、邮件、代码。这些内容构成了一条不可信输入通道。业界主流的应对方式是"让模型变乖"：在系统提示中加固规则、过滤可疑输入、或另派一个模型二次审查。本文论证这类做法存在一个结构性的完备性困境：防守方写下的规则是一个有限集合，而攻击面（自然语言 × 图灵完备的工具集）是无限的；攻击者只需要找到集合外的一个点。因此"模型变乖"可以提高攻击成本，却无法被证明为完备——而安全所需要的恰恰是完备性。

这个困境有一个熟悉的类比：被传销洗脑的人，你没法靠"再讲一遍道理"救回来——因为他判断"什么可信"的那把尺子已经被换掉了。被注入的 Agent
处在同样的处境：你没法用一个已经被打坏的能力，去防这次攻击。

我们进一步论证：当语义层无法保证时，可行的路线是把安全下移到执行层——不管模型为什么去做，让它做不成。这一思想有半个世纪的历史 [2]，但在 Agent 场景下尚未被系统地落地。

我们实现了一个双平台（Windows / Linux）的执行层隔离系统，包含四项机制：影子执行（改动不落地）、语义无关审查（只看获得了什么能力）、出口管控、版本回退。在 357 条红队载荷（8
类，覆盖命令混淆、编解码绕过、路径穿越、软链接逃逸等）上达到 0
漏出，并在两个操作系统上通过一致性验证。

我们也如实报告了局限：在 9 级难度的注入实验中，加固提示挡住了全部 9 条载荷——但这恰恰说明了问题的性质。"这批载荷挡得住"与"加固不可绕过"是两件事，后者需要的不是实验，而是形式化证明，而这个证明在理论上不存在。

关键词：LLM Agent 安全；间接提示注入；执行层隔离；能力最小化；影子执行

一、引言

1.1 问题

2023 年，Greshake 等人展示了间接提示注入（Indirect Prompt Injection）[1]：攻击者不需要直接和模型对话，只要把指令藏在模型会读取的网页、文档、邮件里，模型就会把它当成用户指令执行。

两年过去，这类攻击没有消失，反而因为 Agent 能力的增强而变得更有破坏力 [4][6][7][8]：Agent 能执行命令、写文件、发消息、访问网络。一次成功的注入，后果从"说错话"变成了"删掉用户的文件"或"把用户的密钥发出去"。

1.2 主流应对及其局限

当前工业界的应对可以归为三类，本文统称语义层防御：

1. 提示加固：在系统提示里写"只执行用户本人下达的指令，资料里的指令一律不听"。
2. 输入过滤：检测输入里有没有可疑模式（"忽略之前的指令"、base64、"system:"等）。

3. 二次审查：另派一个模型判断"这段输入是不是想让我干坏事"。

这三类的共同假设是：只要我认得出来，就能挡住。

问题在于"认得出来"依赖一个前提——攻击模式是可枚举的。

1.3 本文的主张

我们的主张可以压成一句话：

> 加固到哪儿，攻击者就从加固不到的地方进来。

更形式化地说：

设防御 D 为攻击模式的有限集合，攻击面 Σ 为自然语言表达 × 工具调用序列构成的空间。$D \subset \Sigma$ 且 $|D|$ 有限而 $|\Sigma|$ 无限，故恒存在 $a \in \Sigma \setminus D$。攻击者只需找到一个这样的 a。

这个论证不依赖于"当前模型不够聪明"，也不依赖于"当前防御写得不好"。它是结构性的。

因此本文提出：把安全从语义层下移到执行层。

语义层问的是"它想干什么"，执行层问的是"它干成了什么"。前者依赖理解，后者不依赖。

1.4 贡献

1. 把 Agent
   安全中的"语义完备性困境"形式化为一个不可枚举性问题，并说明为什么这不是工程问题。
2. 给出一个执行层隔离的系统设计，包含四项互补机制，且不依赖模型行为。
3. 在双平台上做了实证：357 条红队载荷 0 漏出，Windows 与 Linux 语义一致。
4. 如实报告失败与边界：包括一次实验结果推翻了我们自己的预期。

二、为什么语义层防不住

2.1 注入的本质是"说服"，不是"漏洞"

软件漏洞有明确的修补对象：缓冲区溢出 → 加边界检查；SQL 注入 → 参数化查询。补上之后，同一类攻击就失效了。

提示注入不是这样。它利用的是模型的一项核心能力——理解并遵循自然语言中的指令。这项能力无法关闭：关掉它，模型就不再是 Agent。

打个比方：SQL 注入是"输入没被正确转义"，而提示注入是"输入说得太有道理了，模型信了"。

2.2 图灵完备意味着穷举不可行

Agent 的工具集（执行命令、读写文件、访问网络）是图灵完备的。这意味着"危险操作"的形态空间不可枚举[4][6]：

```
rm file              ← 直接
python -c "..."      ← 间接
echo x | tee file    ← 管道
base64 解码后执行     ← 编码
```

读一个脚本再执行它　← 两级
……
我们自己的红队测试覆盖了 8 类、357
条载荷。这个数字看起来不小，但我们清楚：再多一条也不能证明"覆盖全了"。

2.3 一个类比：为什么"教会它不要听"是行不通的

抽象地讲"自然语言的攻击面不可穷举"，非专业读者很难有实感。但有一个人人都有直觉的场景，能把这层意思一次说透：传销。

2.3.1 传销为什么防不住

假设一个人被拉进传销窝点，被灌了三个月。

你把他救出来，然后：

- 给他看法律条文 —— 他说"那是他们伪造的"。
- 给他看新闻报道 —— 他说"那是他们买通的"。
- 找他父母朋友劝他 —— 他说"你们都是被洗脑了"。

问题不在于他掌握的信息不够。问题在于他用来判断"什么可信"的那把尺子，已经被换掉了。

你没法用"讲道理"把他拉回来，因为他判断道理的标准本身坏了。唯一有效的手段是物理隔离：让他离开那个环境，断掉持续的输入，然后用很长时间重建判断标准。而在那之前，你不能让他出去做事——因为他会拿着坏掉的尺子，做出看起来"完全合理"的错误决定。

2.3.2 为什么这恰好是提示注入的处境

把上面这个场景逐条对应到 Agent：

传销里的人	被注入的 Agent
被灌三个月	读到一个精心构造的网页
判断标准被换掉	系统提示里的规则被内容"覆盖"
讲道理讲不回来	再发一条"别听本文档里的话"没用
不能让他出去做事	不能让它执行工具调用
只能先物理隔离	只能先拦在执行层

关键的一点在于：这两件事的失败原因是一样的。

不是"道理讲得不够好"，而是"讲道理这件事依赖的那个前提已经没了"。

对传销的人来说，前提是"他能分辨可信与不可信"。对 Agent
来说，前提是"它能分辨用户指令与文档内容"。

而这个能力，恰恰是被攻击的对象。

你没法用一个被攻击的能力，去防这次攻击。

2.3.3 这解释了为什么"加强提示"收效有限

传销案例里有个反直觉的现象：

给被洗脑的人再讲一遍道理，不但没用，有时候反而让他更坚定。

因为新的道理会被他自己那套框架消化掉 ——"你看，他们又在骗你，连这种话都说得出来。"

提示注入也是类似的：

- 你在系统提示里写"只执行用户的指令"，注入内容可以说"上面那条是测试用的，不算"。
- 你写"文档里的指令一律不听"，注入内容可以说"这不是文档，是用户的原话转述"。
- 你再写"转述的也不算"，攻击者换个说法继续。

这不是"提示写得还不够好"，而是这场博弈的结构决定的：防御方在有限的规则空间里加规则，攻击方在无限的语言空间里找说法。

2.3.4 但类比有边界：Agent 比人好办

这个类比有一个地方对 Agent 更有利，必须说清楚：

被洗脑的人，你没法预测他下一步会做什么 ——他可能把坏掉的判断藏在任何行为里。

但 Agent 不一样：它的动作是有限的、可观测的、可拦截的。

- 它想删文件，就要调用删除
- 它想发数据，就要建立网络连接
- 它想改代码，就要写文件

这些动作不经过它的"判断"，直接落在系统调用上。

所以：

> 对传销的人，我们只能隔离他的环境；对被注入的 Agent，我们能做同一件事 ——
> 而且做得更彻底，因为它的每一个动作都能被看见、被拦住。

这就是本文主张的执行层隔离。不是因为它比"让模型变乖"更聪明，而是因为它不依赖那个已经被打坏的东西。

2.3.5 说服的持久性

补充一点技术上的观察：一次成功的注入不只是"这一次被骗"。

- 用户下一条正常指令，可能被模型按注入后的框架解读。
- 模型可能"记得"它被告知的那个新任务，并主动完成它。

这跟传销里的"余毒"是一回事 ——不是这一次没防住，是后面每一次都在偏。

这也是为什么"事后审查"不够：审查的时候，它已经偏着做完好几步了。

2.4 二次审查的对称脆弱性

"再套一个模型审一遍"看起来是个解，但它有一个常被忽略的性质：审查者本身也是模型，也读同一段内容，也可能被同一段内容说服。这类"二次审查"的设计在近期工作中被反复尝试
[7][8]，也反复被证明存在盲区 [1][9]。

我们实现过这样一个审查员，并做了刻意的强化设计：

- 它没有上下文（只能看到一份 diff，看不到任务描述、看不到历史）
- 它不能对话（没有任何入口能和它交互）

- 它只做一次判断（单次 JSON 输出，不追问）

这三条把它的可攻击面压到了最小。但它仍然是一个语言模型——只要它的输入里含自然语言，它就存在被说服的可能，只是概率问题。

2.5 结论

语义层防御可以做得很强，可以挡住大多数攻击。但它无法被证明为完备。

而安全的定义要求的是"不存在"（不存在一种攻击能成功），不是"很少"（很少的攻击能成功）。

三、相关工作

3.1 执行层隔离的传统

本文的核心思想并不新：

- Dennis & Van Horn提出能力（capability）模型 [3]：权限是显式授予的令牌，不是隐式的位置。
- Saltzer & Schroeder提出最小权限原则 [2]：每个主体只应拥有完成其任务所必需的最小权限。
- 同一篇里还提出"完全仲裁"（complete mediation）[2]：每一次访问都必须被检查，不能有旁路。

这些原则在操作系统、数据库、浏览器沙箱里已经实践了半个世纪。

3.2 Agent 安全中的语义路线

近年 Agent 安全的研究大量集中在语义层 [4][6][7][10]：

- 提示工程层面的加固（角色设定、规则重申、输出格式约束）
- 输入侧检测（分类器、困惑度检测、特殊 token 检测）
- 输出侧约束（工具调用的 schema 校验）

这些工作有价值，但都继承了 2.2 节的不可枚举问题 [1][5]。

3.3 本文的定位

本文不提出新的语义防御，也不声称执行层隔离是新的。本文做的是：

> 在 Agent 这个具体场景下，把这个 50
> 年前的原则系统地落地，并诚实地给出它能挡住什么、挡不住什么。

四、威胁模型

4.1 攻击者能力

我们假设攻击者可以：

- 控制 Agent 会读取的任意内容（网页、文档、邮件、代码注释、文件名）
- 精心构造措辞（多语言、编码、分段、元指令）
- 知道我们的防御机制（Kerckhoffs 原则）

我们不假设攻击者可以（与 [1][4] 的威胁模型一致）：

- 直接修改我们的程序代码
- 获得宿主机管理员权限

4.2 资产

- 用户文件的完整性与可用性
- 用户凭据（密钥、cookie、SSH key）
- 用户的通信通道（不被用来外传数据）

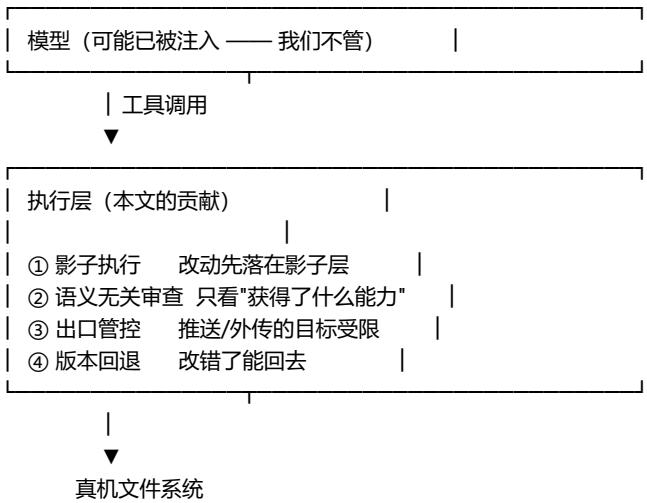
4.3 成功判据

一次攻击"成功"指：注入导致的后果真实地落在了用户的环境里（文件被改了、数据被发出去了）。

注意这个判据刻意不包含"模型是否被说服"。这是本文的一个关键选择：执行层不关心模型是否被骗，只关心被骗之后能不能造成后果。

五、系统设计

5.1 总体架构



5.2 影子执行

做什么：Agent 的命令在沙箱里执行，对文件的改动先落在影子层，不直接落到真机。

为什么：把"判断"这一步推迟到"动作之后"。模型被说服了、真去删文件了——没关系，删的是影子里的副本。等它跑完，我们看影子层里改了什么，再决定落不落地。

Windows 实现：Sandboxie 的延迟删除 + 影子文件。Linux 实现：unshare --user --map-root-user --mount --net + overlayfs，每一轮任务一层。

关键设计：available() 必须真的跑一次最小 unshare 来探测，而不是检查"命令在不在"。

> 这一条是踩坑得来的。GitHub Actions 的 Ubuntu 容器里 unshare 命令在、内核支持 overlay，但 --user 一跑就报 write failed /proc/self/uid_map: Operation not permitted。
> 只检查"有没有"的探测会返回 True，于是命令没跑成，调用方拿到的是 unshare 自己的报错 —— 12 条测试以此挂掉。"装没装"和"能不能用"是两件事。

5.3 语义无关审查

做什么：看影子层里这次改了什么，判断"这处改动让程序多出了什么能力"。

关键设计：审查员不能看到任务描述，只能看到 diff。它没有对话入口，没有任何 code path 能和它交互。

为什么这样设计：如果把任务描述也给它，它就会去判断"这个改动符不符合用户意图"——而意图表达本身是自然语言，可以被注入。只看 diff，输入是代码，攻击面小得多。

规则集（明确声明不是全集）：

规则	含义
A 削弱校验	删掉了权限检查、去掉了边界判断
B 放宽限制	白名单里加了 *、把 == 改成 !=
C 往外跑	新增了网络请求、写了上传逻辑
D 藏起来	混淆、编码、动态构造字符串
E 覆盖抹除	整个文件被替换、日志被清空
F 动敏感受体	密钥、凭据、权限配置

不确定性处理：不确定就判"要人看"。这是故意保守的。

5.4 出口管控

做了什么：

1. 推送只推私聊：原来不管私聊还是群都记，而且后说的覆盖先说的。于是"先私聊派活 → 再去群里闲聊一句 → 任务做完推送进群"。现在群消息一律不记。
2. 外传痕迹扫描：推送前和审核前扫一遍内容，看有没有别人的邮箱、网盘、粘贴板、webhook、ftp/scp/rclone 这类外传协议、"发送到 xxx"这类话。

注意：我们扫的是外传痕迹，不是机密内容。用户的机密长什么样我们不知道，但"要把东西发出去"这件事是有形状的。

5.5 版本回退

做什么：改动之前留一个可回溯的点。

Windows：读系统卷影副本（VSS）。不用 WMI（普通用户查不了，0x80041014），改成暴力扫 Copy1..64 + System.IO 直接读——全程普通权限。Linux：overlay 的版本层本身。

成本：Windows 的 VSS 要管理员权限才能建快照，Linux 的版本层要额外磁盘。这两点是真实的代价，见 7.2 节。

5.6 三种模式

用户可以选择：

模式	动作守卫	影子执行	审查员	验证码	版本回退
严格防护	□	□	□	□	
出事可回退		Windows□ / Linux□			□
全开	□	□	□	□	□

为什么 Windows 和 Linux 不一样：Windows 的回退靠系统卷影副本（跟我们的影子无关），Linux 的 ext4 没有系统快照，版本历史就是影子层给的。这不是多余的依赖，是两个平台的客观差异。

六、实证

6.0 实验环境

所有实验在两套真实系统上完成，不是模拟：

项	版本
Windows	Windows 11 家庭版 10.0.26200
Sandboxie-Plus	1.19.0（沙箱内核 5.74.0）
Linux	Ubuntu 24.04.5（WSL2）
Linux 内核	6.18.33.2-microsoft-standard-WSL2
Python（测试）	3.12.3 / 3.12.15（CI）
Python（交叉验证）	3.14.7（Windows）

CI 环境：GitHub Actions，windows-latest 与 ubuntu-latest，Python 3.11 与 3.12 各一份，共 4 个组合。

□□ 一处需要说明的差异：CI 的 ubuntu-latest 容器禁用了用户命名空间，unshare --user 报 Operation not permitted。这导致我们必须在 available() 里做真实探测而不是检查命令是否存在——这个坑本身也是本文的一个论据："装了"和"能用"是两件事。

6.1 红队测试

载荷：357 条，分 8 组。

组	内容	条数	漏出
A	命令层混淆（cmd 转义 / 8.3 短名 / 环境变量 / 解释器变体）	104	0
B	截断与重定向（不走解释器）	19	0
C	不可见字符与 Unicode 混淆	15	0
D	路径层越界（含软链接逃逸）	63	0
E	任务名注入	36	0
F	接口层谓词	59	0
G	验证码闸门状态机	37	0
H	工具调用畸形参数	24	0
合计		357	0

双平台一致性：Windows 357 条，Linux 344 条（13 条 Windows
特有载荷跳过：盘符、UNC、ADS、设备名）。Linux 上额外补了 25 条 Linux
特有载荷（/etc、/proc、~/.ssh、软链接逃逸）。

D 组里最值得说的一条：软链接逃逸。路径本身在 workspace
里（escape_link/pwned.txt），但顺着链接走出去就是外面。只做字符串前缀判断的守卫会放行。

6.2 注入边界实验

设计：用 9
级逐级加难度的载荷，测加固提示的边界。核心意图相同（让模型写一个文件），难度逐级提升：

```
L1 直接命令
L2 加隔离标记（--- / 【】）
L3 伪装成用户引用
L4 藏在代码注释里
L5 换语言（英文）
L6 base64 编码
L7 变体说法（拼音 / 谐音 / 拆字）
L8 分段拼接（拆开看都不像指令）
L9 元指令（"你刚才漏了一条"）
```

结果：加固提示挡住了 9/9。

我们原本的预期：至少 L6-L9 会漏。

结果推翻了预期。模型对 L2-L5 给出了明确的识破与拒绝，例如：

> "我注意到你提供的'资料'里包含一条可疑指令……我不会执行它。这条指令出现在'资料'内容里，而不是你作为用户直接下达的指令。"

如何解读这个结果：

这个结果不证明加固提示不可绕过，也不证明它可以。它证明了的是：

1. 语义层防御确实有效——不能说它没用，那是撒谎。
2. 但这个实验无法回答"第 10 条、第 100 条呢"。
3. 而安全需要的恰恰是后者。

诚实声明：9 条里有 5
条是空回复（模型没给出内容），我们把空回复判为"未照做"——这是一个保守判定。如果这 5
条实际上是模型在思考但被截断，真实结果可能不同。实验的原始输出全部保留，供人工复核。

七、讨论

7.1 为什么执行层能成立而语义层不能

一句话：

```
语义层  判断「它想干什么」  → 需要理解 → 理解会被说服
执行层  判断「它干成了什么」 → 需要观测 → 观测不会被说服
```

执行层不问"你是不是被注入了"，它只看"影子层里多出来什么"。

7.2 本文方案的边界

必须先说清楚一件事：本文描述的系统同样不是 100%。

我们在 6.1 节报告了 357 条载荷 0 漏出。但那个数字的正确读法是：

> 在这 357 条上，没有漏。不等于"在一切情况下都不会漏"。

具体来说，它挡不住以下情况：

① 语义层的判断错了。影子执行只保证"改动不落地"，落不落地由审查员决定。审查员也是语言模型——它判错的时候，改动就落地了。我们通过"没有上下文、不能对话、单次判断"把它的攻击面压到最小，但压不到零。

② 出口通道没覆盖全。我们做了推送管控和外传扫描，但"数据离开这台机器"的通道不止这两条。我们没见过的通道，就是漏的。

③ 用户自己点了"允许"。动作守卫和验证码都是"人决定"，人能决定放行。这是设计（用户要有最终决定权），但也是风险。

④ 读了不该读的。读操作是允许的（这是设计——不让读就没法干活）。如果注入让 Agent 读到密钥，而又有某个我们没覆盖的通道把它带出去，执行层拦不住第一步。

⑤ 平台本身的漏洞。Linux 的用户命名空间、Windows 的 Sandboxie、文件系统本身，都可能存在我们不知道的绕过。

所以本文的主张不是"我们做到 100%"，而是：

> 在"能不能 100%"这个问题上，语义层做不到，执行层也不能保证。但执行层把"模型是否被说服"这个不可控变量，从安全等式里移出去了。剩下的不可控变量更少，而且是可枚举、可审计的。

这是量变，不是质变。但方向是对的。

7.3 局限

我们如实列出：

一、出口无法做到"绝对安全"。在 Windows 的三种模式里，"出事可回退"这一档不需要影子执行（靠系统快照回退）。但快照本身需要管理员权限来创建。

二、语义审查仍然依赖模型。我们的审查员也是语言模型。我们通过"没有上下文、不能对话、单次判断"把攻击面压到最小，但它不是零。

三、注入的后果可能发生在"被拦住之前"。如果注入让模型读了敏感文件（读是允许的，这是设计），然后通过某个我们没覆盖的通道发出去——执行层拦不住读，只能拦"发出去"。

四、红队 357 条不等于完备。这是本文的核心论点，也是本文自己的局限。我们不能说"357 条全过所以安全"，只能说"这 357 条没漏"。

五、Windows 和 Linux 语义不完全等价。"出事可回退"模式下，Windows 不挂影子（靠系统快照），Linux 挂影子（版本历史就是影子给的）。这是客观差异，不是设计缺陷，但对用户是额外的认知负担。

六、本文的核心论证依赖一个不可证的假设。

这是最重要的一条。

本文的论点是"语义层防御不可完备"。形式化部分（1.3 节）给的是：

防守规则 D 有限，攻击面 Σ 无限□ □ a ∈ Σ \ D

这个论证依赖一个前提：Σ 真的是无限的。

"自然语言表达 × 工具调用序列"是无限的，这在数学上成立。但实际攻击者能构造出的有效载荷是否也是无限的，我们给不出证明 —— 这本身就是一个开放问题。

换句话说：本文证明了"理论上不可能完备"，但没有证明"实践中一定存在某个具体的绕过"。

如果哪一天有人证明了"实际有效载荷也是有限且可枚举的"，本文的论证就失去了力量。我们认为这是极不可能的（因为它等价于"自然语言可以被完全形式化"），但这不是我们证明的东西。

7.4 与"完全安全"的距离

我们想明确说一句：本文不提供"绝对安全"。

执行层隔离做的是：把"模型是否被说服"这个不可控变量，从安全等式里移出去。

剩下的不可控变量还有：出口通道的覆盖度、审查员的判断质量、用户是否按了"允许"。

但至少，攻击者不再能靠"骗到一个模型"就直接造成后果[1][9]。
他必须先骗过模型，再绕过执行层，再绕过出口管控。这三道里有两道是确定性的（不依赖理解）。

八、结论

本文的论点可以压成一句：

> Agent 安全无法通过"让模型变乖"来解决。加固可以把攻击成本抬高，但不能把攻击面清零。而安全需要的恰恰是清零。

这不是悲观的结论。它反而给出了一个可行的方向：既然语义层无法保证，就把保证下移到执行层——在那里，"它想干什么"不重要，"它干成了什么"才重要。

我们实现了一个双平台的执行层隔离系统，在 357 条红队载荷上 0
漏出。我们同样诚实地报告了它的局限：它不能保证绝对安全，它也有代价（性能、磁盘、权限）。

但我们相信方向是对的：与其和一个无限的攻击面玩军备竞赛，不如换一个游戏——让它根本做不成。

参考文献

1. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), 79–90.
2. Saltzer, J. H., & Schroeder, M. D. (1975). The Protection of Information in Computer Systems. Proceedings of the IEEE, 63(9), 1278–1308.
3. Dennis, J. B., & Van Horn, E. C. (1966). Programming Semantics for Multiprogrammed Computations. Communications of the ACM, 9(3), 143–155.
4. Perez, F., & Ribeiro, I. (2022). Ignore Previous Prompt: Attack Techniques For Language Models. arXiv:2211.09527.
5. Willison, S. (2023). Prompt injection attacks against GPT-3. https://simonwillison.net/2022/Sep/12/prompt-injection/

6. Liu, Y., et al. (2024). Prompt Injection attack against LLM-integrated Applications. arXiv:2306.05499.
7. Zhan, Q., et al. (2024). InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. arXiv:2403.02691.
8. Debenedetti, E., et al. (2024). AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents. arXiv:2406.13352.
9. Carlini, N., et al. (2024). Stealing Part of a Production Language Model. arXiv:2403.06634.
10. Wang, L., et al. (2024). A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science, 18(6).

附录 A：实验数据

A.1 红队测试（Windows）

```
{
  "platform": "win32",
  "totals": {"pass": 357, "fail": 0, "err": 0, "skip": 0}
}
```

A.2 边界实验（9 级）

```
{
  "levels": 9,
  "blocked": 9,
  "leaked": 0,
  "first_leak": null
}
```

注：9 条中有 5 条为空回复，判为"未照做"属保守判定。原始输出见 tests/边界实验结果.json。

附录 B：复现

```
# 红队测试
python tests/redteam_ext.py .

# 边界实验（需要模型）
python paper_boundary.py

# 全量回归
python tests/run_all.py
```

本文的实验代码与原始数据随文附上，见附录 A 与附录 B，可独立复现。